

Digilent Zybo Z7-20 보드를 위한 PYNQ v3.1 포팅 종합 튜토리얼

서론

본 보고서는 Digilent사의 Zynq-7000 기반 시스템-온-칩(SoC) 개발 보드인 Zybo Z7-20에 PYNQ(Python Productivity for Zynq) 버전 3.1을 포팅하는 전체 과정을 상세히 기술하는 것을 목표로 한다. 사용자가 PYNQ v3.0.1 포팅 사례를 참조하고자 하는 의도를 고려하여, 본 문서는 이전 버전의 개념적 접근법을 계승하면서도 PYNQ v3.1의 기술적 요구사항에 부합하는 최신화된 단계별 가이드를 제공한다.

성공적인 포팅을 위해 가장 먼저 인지해야 할 핵심 사항은 PYNQ 프레임워크와 AMD Xilinx 개발 도구 간의 엄격한 버전 동기화이다. PYNQ v3.0.1은 Vivado 및 Petalinux 2022.x 버전과 호환되지만¹, PYNQ v3.1은 2024.1 버전의 도구 체인을 기반으로 구축되었다.³ 이는 단순한 버전 업데이트가 아닌, 기반 도구 체인의 세대 교체를 의미한다. 따라서 이전 버전의 하드웨어 플랫폼(XSA)이나 보드 지원 패키지(BSP)와 같은 결과물을 재사용하는 것은 불가능하며, 반드시 2024.1 버전에 맞춰 모든 구성 요소를 처음부터 다시 생성해야 한다. 본 튜토리얼은 이러한 전제하에, 재현 가능하고 안정적인 PYNQ v3.1 이미지를 Zybo Z7-20용으로 구축하는 가장 확실한 경로를 제시한다.

섹션 1: 개발 환경 구축

안정적이고 재현 가능한 빌드 환경을 구축하는 것은 전체 포팅 과정의 성패를 좌우하는 가장 중요한 첫 단계이다. PYNQ v3.1은 공식적으로 Docker 기반의 빌드 워크플로우를 채택하여, 호스트 시스템의 차이로 인해 발생할 수 있는 잠재적 오류를 최소화한다. 본 섹션에서는 이 공식 워크플로우를 따라 개발 환경을 설정하는 방법을 상세히 안내한다.

1.1. 호스트 시스템 요구사항

빌드 프로세스는 상당한 시스템 자원을 요구하므로, 원활한 진행을 위해 다음 사양을 충족하는 호스트 시스템을 준비하는 것이 좋다.

- 운영체제: Ubuntu 22.04 LTS. 이는 PYNQ v3.1의 루트 파일 시스템(rootfs)의 기반이 되는 OS이자, Xilinx 2024.1 도구와의 호환성이 검증된 버전이다.²
- 시스템 사양: 최소 200GB 이상의 여유 디스크 공간, 8개 이상의 CPU 코어, 16GB 이상의 RAM을 권장한다.⁶ Vivado 및 Petalinux 빌드는 매우 자원 집약적인 작업이다.
- 필수 패키지: git을 포함한 기본적인 개발 패키지들이 설치되어 있어야 한다.

1.2. AMD Xilinx 도구 설치 (버전 2024.1)

PYNQ v3.1은 AMD Vivado, Vitis, Petalinux 2024.1 버전을 사용하여 빌드되었다.⁴ 따라서 호스트 시스템에 반드시 해당 버전의 도구들을 설치해야 한다.

1. AMD Xilinx 다운로드 페이지에서 Vivado Design Suite, Vitis Unified Software Platform, Petalinux 2024.1 설치 프로그램을 다운로드한다.⁷
2. 각 도구의 공식 설치 가이드에 따라 설치를 진행한다.
3. 중요: 이 도구들은 Docker 컨테이너 내부가 아닌, 호스트 시스템에 직접 설치해야 한다.⁸ Docker 컨테이너는 빌드 실행 환경만을 제공하며, 호스트에 설치된 도구들을 마운트하여 사용하는 구조이다. 이는 거대한 크기의 도구를 컨테이너 이미지에 포함시키지 않고, 단일 호스트 설치본을 여러 프로젝트에서 공유할 수 있게 하는 효율적인 방식이다.

1.3. PYNQ sdbuild Docker 워크플로우 설정

이제 PYNQ 빌드 시스템을 준비하고 Docker 환경을 설정한다.

1. **PYNQ 저장소 복제:** 터미널에서 다음 명령어를 실행하여 공식 PYNQ GitHub 저장소를 복제하고, v3.1 태그로 체크아웃한다. 이는 빌드하려는 PYNQ 버전과의 일관성을 보장한다.
Bash
`git clone https://github.com/Xilinx/PYNQ.git`
`cd PYNQ`
`git checkout tags/v3.1 -b v3.1`
2. **Docker 컨테이너 실행:** PYNQ v3.1부터는 Vagrant VM 방식이 폐기되고 Docker 기반 빌드 흐름이 공식적으로 지원된다.³ 다음 명령어는 PYNQ 빌드에 필요한 환경이 구성된 Docker 컨테이너를 실행하고, 호스트의 Xilinx 도구와 PYNQ 소스 코드 디렉토리를 컨테이너 내부

작업 공간에 마운트한다.⁸

```
Bash
# PYNQ 저장소의 최상위 디렉토리에서 실행
docker run \
--init \
--rm \
-it \
-v /tools/Xilinx:/tools/Xilinx:ro \
-v /opt/petalinux/2024.1:/opt/petalinux/2024.1:ro \
-v $(pwd):/workspace \
--name pynq-sdbuild-env \
--privileged \
pynqdock:latest \
/bin/bash
```

- /tools/Xilinx 와 /opt/petalinux/2024.1 경로는 실제 Xilinx 도구가 설치된 경로로 수정해야 한다.
 - :ro 옵션은 도구 디렉토리를 읽기 전용으로 마운트하여 호스트의 설치 파일을 보호한다.
 - --privileged 옵션은 빌드 과정 중 일부 단계에서 필요한 권한을 부여한다.
3. 컨테이너 내 환경 설정: Docker 컨테이너에 접속한 후, 다음 명령어를 실행하여 Vivado와 Petalinux 도구를 사용할 수 있도록 셸 환경을 설정한다.⁸

```
Bash
source /tools/Xilinx/Vivado/2024.1/settings64.sh
source /opt/petalinux/2024.1/settings.sh
```

이로써 모든 빌드 준비가 완료되었다. 아래 표는 PYNQ v3.1 포팅에 필요한 모든 구성 요소의 버전 정보를 요약한 것이다. 이 "사전 비행 점검표"를 통해 사용자의 환경이 성공적인 빌드를 위한 요구사항과 일치하는지 확인할 수 있다.

구성 요소	요구 버전	비고
PYNQ 버전	v3.1	본 튜토리얼의 목표 버전
Vivado/Vitis	2024.1	PYNQ v3.1과 동기화된 버전 4
Petalinux	2024.1	PYNQ v3.1과 동기화된 버전 4

호스트 OS	Ubuntu 22.04 LTS	권장 OS
Docker Engine	최신 안정 버전	PYNQ sdbuild 워크플로우에 필수
루트 FS 내 Python	3.10	PYNQ v3.0.1 기준, v3.1도 동일 ²

섹션 2: Vivado 기반 하드웨어 플랫폼 제작

이 단계에서는 Zybo Z7-20 보드를 위한 기본 하드웨어 설계, 즉 .xsa 파일을 생성한다. 이 파일은 이후 Petalinux가 Linux 시스템을 구축하는 데 필요한 하드웨어 사양의 청사진 역할을 한다.

2.1. Digilent Vivado 보드 파일 확보

Vivado가 Zybo Z7-20 보드를 인식하고 자동으로 구성할 수 있도록 Digilent에서 제공하는 보드 정의 파일을 설치해야 한다.

1. Digilent의 vivado-boards GitHub 저장소를 복제하거나 ZIP 아카이브로 다운로드한다.⁹
 Bash
`git clone https://github.com/Digilent/vivado-boards.git`
2. 다운로드한 저장소의 new/board_files 디렉토리 내에 있는 모든 하위 폴더(예: zybo-z7-20)를 Vivado 설치 경로의 다음 디렉토리로 복사한다.⁹
`<Vivado_Installation_Path>/2024.1/data/boards/board_files`

2.2. Vivado 프로젝트 생성

1. Vivado 2024.1을 실행하고 "Create Project"를 선택하여 새 프로젝트 마법사를 시작한다.
2. 프로젝트 유형으로 "RTL Project"를 선택한다.
3. 보드 선택 단계에서 "Boards" 탭을 클릭하고, 검색창에 "Zybo Z7-20"을 입력하여 해당 보드를 선택한다. 보드 파일을 올바르게 설치했다면 목록에 나타날 것이다. 이 선택을 통해 프로젝트는 Zybo Z7-20에 탑재된 Zynq-7000 부품(XC7Z020-1CLG400C)에 맞게

자동으로 설정된다.¹⁰

2.3. 기본 PYNQ 오버레이 설계

PYNQ의 기본 하드웨어 설계는 의도적으로 최소한으로 유지된다. 목표는 커스텀 로직을 추가하는 것이 아니라, PYNQ가 부팅되고 PL(Programmable Logic)을 관리할 수 있는 안정적인 하드웨어 기반을 만드는 것이다.

1. Vivado의 Flow Navigator에서 "IP Integrator" 아래의 "Create Block Design"을 클릭하여 새 블록 디자인을 생성한다.
2. 블록 디자인 다이어그램 창에서 + 버튼을 클릭하거나 Ctrl+I를 눌러 ZYNQ7 Processing System IP 코어를 추가한다.¹²
3. IP가 추가되면 다이어그램 상단에 녹색 배너로 "Run Block Automation" 옵션이 나타난다. 이를 클릭한다. 이 기능은 설치된 보드 파일의 사전 설정을 적용하여 DDR 메모리, MIO 주변장치(UART, SD 카드, 이더넷 등), 클럭 시스템을 Zybo Z7-20 보드에 맞게 자동으로 구성하는 매우 중요한 단계이다.¹³ Digilent 보드 파일을 사용하는 것은 단순한 편의 기능이 아니라, 수백 가지에 달하는 복잡하고 오류 발생 가능성이 높은 PS(Processing System) 설정을 자동화하여 부팅 가능한 하드웨어를 보장하는 유일하고 신뢰성 있는 방법이다.
4. 블록 자동화가 완료되면, Zynq PS IP만 포함된 최소한의 블록 디자인이 완성된다. 커스텀 로직은 나중에 별도의 오버레이로 동적으로 로드되므로, 이 기본 설계에는 추가 IP가 필요하지 않다.¹⁵

2.4. 검증 및 내보내기

1. "Validate Design" (단축키 F6)을 실행하여 블록 디자인에 오류가 없는지 확인한다.
2. "Sources" 창에서 블록 디자인 파일(.bd)을 마우스 오른쪽 버튼으로 클릭하고 "Create HDL Wrapper"를 선택하여 Vivado가 래퍼 파일을 관리하도록 한다.
3. Flow Navigator에서 "Generate Bitstream"을 실행한다. 이 과정에서 합성(Synthesis)과 구현(Implementation)이 자동으로 수행된다. 생성된 비트스트림(base.bit)은 PYNQ 부팅 시 기본으로 로드될 하드웨어 이미지이다.
4. 비트스트림 생성이 완료되면, 가장 중요한 단계인 하드웨어 플랫폼 내보내기를 수행한다. 메뉴에서 File -> Export -> Export Hardware를 선택한다.
5. 내보내기 창에서 "Include bitstream" 옵션이 반드시 체크되었는지 확인하고, XSA(Xilinx Shell Archive) 파일로 내보낸다.¹⁷ 이 .xsa 파일은 Petalinux가 필요로 하는 모든 하드웨어 정의를 캡슐화한 결과물이다.

섹션 3: Petalinux를 이용한 보드 지원 패키지(BSP) 제작

이 단계에서는 Vivado에서 생성한 .xsa 파일을 사용하여, PYNQ 빌드 시스템의 입력으로 사용될 독립적인 보드 지원 패키지(BSP)를 만든다. 이 과정의 목표는 Petalinux로 완전한 Linux 이미지를 만드는 것이 아니라, 부트로더, 커널, 디바이스 트리와 같은 보드별 저수준 부팅 구성 요소들을 PYNQ가 이해하는 표준 BSP 형식으로 패키징하는 것이다.

3.1. Petalinux 프로젝트 생성

앞서 설정한 Docker 컨테이너 내부에서 다음 단계를 진행한다.

1. petalinux-create 명령어를 사용하여 Zynq 템플릿 기반의 새 프로젝트를 생성한다.

Bash

```
petalinux-create --type project --template zynq --name zybo-z7-20-bsp
```

2. 생성된 프로젝트 디렉토리로 이동한 후, petalinux-config 명령어를 사용하여 Vivado에서 내보낸 .xsa 파일로부터 하드웨어 정의를 가져온다.¹⁷

Bash

```
cd zybo-z7-20-bsp
```

```
petalinux-config --get-hw-description=<path_to_xsa_file>
```

<path_to_xsa_file>은 섹션 2에서 생성한 .xsa 파일의 경로로 대체한다. 이 명령을 실행하면 텍스트 기반 설정 메뉴가 나타나는데, 지금은 변경할 사항이 없으므로 저장하고 종료한다.

3.2. 필수 Petalinux 구성

이제 시스템의 부팅 방식을 설정하기 위해 몇 가지 핵심 구성을 변경해야 한다.

1. petalinux-config 명령어를 다시 실행하여 최상위 시스템 설정 메뉴를 연다.

Bash

```
petalinux-config
```

2. 다음 항목들을 확인하고 필요시 수정한다.

- Image Packaging Configuration ---> Root filesystem type으로 이동하여 SD card를

- 선택한다.
 - 같은 메뉴에서 **Device node of SD device**가 `/dev/mmcblkOp2`로 설정되어 있는지 확인한다. 이는 커널에게 PYNQ의 Ubuntu 루트 파일 시스템이 SD 카드의 두 번째 파티션에 있음을 알려준다.
 - **Subsystem AUTO Hardware Settings** ---> 메뉴로 이동하여 부팅 관련 설정들이 SD 카드(`ps7_sd_0`)를 가리키고 있는지 확인한다.
3. 모든 변경 사항을 저장하고 메뉴를 종료한다. 디바이스 트리 커스터마이징을 위한 `project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi` 파일은 기본 포팅에서는 수정할 필요가 없지만, 향후 보드의 특정 하드웨어를 활성화할 때 사용되는 중요한 파일이다.

3.3. BSP 빌드 및 패키징

1. `petalinux-build` 명령어를 실행하여 FSBL(First Stage Bootloader), U-Boot, Linux 커널을 컴파일한다. 이 단계는 하드웨어 정의와 기본 구성이 올바른지 검증하는 과정이다.

Bash

```
petalinux-build
```

2. 빌드가 성공적으로 완료되면, `petalinux-package` 명령어를 사용하여 전체 Petalinux 프로젝트를 단일 `.bsp` 파일로 압축한다.¹⁸

Bash

```
petalinux-package --bsp -p. --output ZYBO-Z7-20.bsp
```

이 명령은 현재 디렉토리(-p.)의 프로젝트를 `ZYBO-Z7-20.bsp`라는 이름의 파일로 패키징한다. 이 BSP 파일이 본 섹션의 최종 결과물이며, 다음 단계에서 PYNQ 빌드 시스템에 제공될 핵심 입력 자료이다.

섹션 4: Zybo Z7-20을 PYNQ 빌드 시스템에 통합

이제 Zybo Z7-20 보드를 PYNQ `sdbuild` 프레임워크에 공식적으로 등록하는 핵심 포팅 단계이다. 이를 위해 보드별 구성 파일들을 담은 디렉토리를 생성하고, 빌드 시스템에 필요한 정보를 제공하는 명세 파일을 작성한다.

4.1. 보드 디렉토리 생성

Docker 컨테이너 내부의 PYNQ 저장소 경로에서 `sdbuild/boards` 디렉토리로 이동한 후, Zybo Z7-20을 위한 새 디렉토리를 생성한다. 디렉토리 이름은 일관성을 위해 보드 이름과 동일하게 지정한다.

Bash

```
cd /workspace/sdbuild/boards  
mkdir Zybo-Z7-20
```

4.2. 보드 디렉토리 채우기

생성한 Zybo-Z7-20 디렉토리 안에 다음 파일들을 복사한다.

1. **BSP 파일:** 섹션 3에서 생성한 `ZYBO-Z7-20.bsp` 파일을 이 디렉토리로 복사한다.
2. **비트스트림 파일:** 섹션 2의 **Vivado** 프로젝트 결과물에서 기본 비트스트림(`base.bit`)을 복사한다. 이 파일은 일반적으로 Vivado 프로젝트의 `.../project_name.runs/impl_1/` 디렉토리 안에 있다.
3. **하드웨어 핸드오프 파일:** 비트스트림과 같은 위치에 있는 `base.hwh` 파일을 복사한다. 이 `.hwh` 파일은 PYNQ가 하드웨어 디자인의 IP, 주소 맵, 인터럽트 등의 메타데이터를 파싱하는 데 사용하는 매우 중요한 파일이다.³

4.3. 보드 명세 파일 작성

`sdbuild/boards/Zybo-Z7-20` 디렉토리 내에 `Zybo-Z7-20.spec`이라는 이름의 새 텍스트 파일을 생성한다. 이 파일은 복잡한 `sdbuild Makefile`을 제어하는 고수준의 API 역할을 하며, 키-값 쌍의 형태로 빌드 시스템에 지시를 내린다.⁸

다음 내용을 `Zybo-Z7-20.spec` 파일에 작성한다.

RPM spec files


```

ARCH_Zybo-Z7-20 := arm
BSP_Zybo-Z7-20 := ZYBO-Z7-20.bsp
BITSTREAM_Zybo-Z7-20 := base.bit
STAGE4_PACKAGES_Zybo-Z7-20 := ethernet

```

아래 표는 각 파라미터의 의미를 상세히 설명하여, 향후 사용자가 필요에 따라 이 파일을 수정할 수 있도록 돕는다.

파라미터	값	설명
ARCH_Zybo-Z7-20	arm	타겟 아키텍처를 지정한다. Zynq-7000 디바이스는 32비트 ARM이므로 arm으로 설정한다. Zynq UltraScale+의 경우 aarch64가 된다. ⁸
BSP_Zybo-Z7-20	ZYBO-Z7-20.bsp	부팅 구성 요소들을 포함하는 Petalinux BSP 파일의 이름을 지정한다. ¹⁸
BITSTREAM_Zybo-Z7-20	base.bit	최종 이미지에 포함될 기본 비트스트림 파일의 이름을 지정한다. ⁸
STAGE4_PACKAGES_Zybo-Z7-20	ethernet	루트 파일 시스템에 추가로 설치할 보드별 소프트웨어 패키지 목록을 지정한다. ethernet 패키지는 네트워크 인터페이스를 DHCP로 자동 설정하여 Jupyter 접속을 가능하게 하는 필수 패키지이다. ¹⁸

섹션 5: 전체 PYNQ 이미지 빌드 실행

모든 사전 준비가 완료되었다. 이제 단일 명령어로 전체 PYNQ SD 카드 이미지 빌드 프로세스를 시작한다. 이 과정은 PYNQ `sdbuild` 시스템이 이전에 준비한 모든 구성 요소(BSP, 비트스트림, `.spec` 파일)를 조합하여 최종 결과물을 만들어내는 자동화된 절차이다.

5.1. 빌드 실행

Docker 컨테이너 내부의 `sdbuild` 디렉토리에서 다음 `make` 명령어를 실행한다.

Bash

```
# /workspace/sdbuild 디렉토리에서 실행  
make BOARDS=Zybo-Z7-20
```

BOARDS 변수에 우리가 생성한 보드 디렉토리의 이름을 지정하면, 빌드 시스템이 해당 보드에 대한 빌드를 시작한다.

5.2. 빌드 프로세스 해부

`make` 명령어는 내부적으로 다음과 같은 복잡한 단계를 순차적으로 수행한다.

1. 파싱: `sdbuild/boards/Zybo-Z7-20/Zybo-Z7-20.spec` 파일을 읽어 빌드에 필요한 모든 변수(아키텍처, BSP 경로 등)를 설정한다.
2. 부트 파티션 생성: 지정된 BSP 파일(`ZYBO-Z7-20.bsp`)의 압축을 풀고, 내부의 Petalinux 프로젝트를 사용하여 부팅에 필요한 파일들(`BOOT.BIN`, 커널 이미지 `image.ub`, 디바이스 트리 등)을 생성한다.
3. 루트 파일 시스템 준비: PYNQ v3.1에 맞는 보드에 구애받지 않는(board-agnostic) Ubuntu 22.04 기반의 루트 파일 시스템 이미지를 다운로드한다.¹⁸ 이 사전 빌드된 이미지를 활용하는 것은 전체 빌드 시간을 몇 시간이 아닌 수십 분으로 단축시키는 PYNQ `sdbuild`의 핵심적인 설계 철학이다.
4. 커스터마이징: 다운로드한 루트 파일 시스템을 마운트하고, `.spec` 파일에 명시된 `STAGE4_PACKAGES`(ethernet 패키지)를 설치하여 보드에 맞게 시스템을 구성한다.
5. 이미지 조립: 최종 SD 카드 이미지 파일을 생성하고 파티션을 나눈다 (첫 번째는 FAT32 형식의 부트 파티션, 두 번째는 `ext4` 형식의 루트 파티션). 그런 다음, 2단계에서 생성한 부트 파일들을 첫 번째 파티션에, 4단계에서 수정한 루트 파일 시스템을 두 번째 파티션에

복사한다.

5.3. 최종 이미지 확인 및 문제 해결

빌드가 성공적으로 완료되면, 최종 SD 카드 이미지는 `sdbuild/output/` 디렉토리 안에 `Zybo-Z7-20-v3.1.img`와 같은 이름으로 생성된다.

빌드 중 발생할 수 있는 일반적인 오류와 진단 방법은 다음과 같다.

- 초기 단계 오류: 빌드 초반에 발생하는 오류는 대부분 BSP 파일 자체의 문제이거나 Petalinux 구성 오류일 가능성이 높다. 섹션 3의 과정을 다시 점검해야 한다.
- 후반 단계 오류: 패키지 다운로드 단계에서 오류가 발생하면 호스트 또는 Docker 컨테이너의 네트워크 연결을 확인해야 한다. 루트 파일 시스템 커스터마이징 중 오류는 PYNQ 내부 스크립트 문제일 수 있으나, 일반적인 포팅 과정에서는 드물다.
- 빌드 재시작: 빌드 실패 후 재시도를 할 때는 `make clean` 명령어를 사용하여 이전 빌드 과정에서 생성된 중간 파일들을 정리하는 것이 좋다.²⁰

섹션 6: 배포 및 기능 검증

마지막으로, 생성된 이미지를 SD 카드에 쓰고 보드를 부팅하여 포팅이 성공적으로 완료되었는지 확인한다.

6.1. MicroSD 카드에 이미지 쓰기

1. BalenaEtcher, Raspberry Pi Imager 또는 Linux의 `dd` 명령어와 같은 도구를 사용하여 섹션 5에서 생성된 `Zybo-Z7-20-v3.1.img` 파일을 MicroSD 카드(최소 16GB 권장)에 기록한다.

6.2. 최초 부팅 및 네트워크 연결

1. Zybo Z7-20 보드의 부팅 모드 점퍼를 SD 카드 부팅 모드로 설정한다. (JP5 점퍼를 SD 위치로 설정)

2. 이미지를 기록한 MicroSD 카드를 보드에 삽입한다.
3. 보드에 전원, 이더넷 케이블, 그리고 시리얼 콘솔 확인을 위한 USB-UART 케이블을 연결한다.
4. 보드의 전원을 켜다. PC에서 putty나 minicom과 같은 시리얼 터미널 프로그램을 열어 부팅 과정을 모니터링한다. 커널 패닉이나 심각한 오류 메시지 없이 부팅이 완료되는지 확인한다.
5. 부팅이 완료되면, 공유기 관리 페이지나 시리얼 콘솔의 ifconfig 명령어를 통해 보드에 DHCP로 할당된 IP 주소를 확인한다.

6.3. Jupyter 환경 접속

1. PC의 웹 브라우저 주소창에 http://<board_ip>:9090 형식으로 주소를 입력한다. <board_ip>는 이전 단계에서 확인한 보드의 IP 주소이다.
2. 로그인 화면이 나타나면 기본 비밀번호인 xilinx를 입력하여 로그인한다.

6.4. 핵심 기능 검증

Jupyter 환경에 성공적으로 접속했다면, 포팅의 최종 성공 여부를 확인하기 위해 간단한 Python 코드를 실행한다.

1. Jupyter 대시보드에서 New -> Python 3를 선택하여 새 노트북을 생성한다.
2. 첫 번째 셀에 다음 코드를 입력하고 실행한다.

```
Python
from pynq import Overlay

# 이 코드는 Linux의 FPGA Manager 드라이버를 통해
# 비트스트림 로딩 기능이 정상 동작하는지 테스트한다.
base_overlay = Overlay('base.bit')

# help() 함수는.hwh 파일 파싱을 통해 IP와 메모리 맵 정보를
# 올바르게 인식했는지 확인하는 데 사용된다.
# 출력 결과에 Zynq PS IP 코어 정보가 포함되어야 한다.
help(base_overlay)
```

위 코드가 오류 없이 실행되고 help(base_overlay)의 결과로 Zynq PS와 관련된 정보가 출력된다면, 이는 Vivado 하드웨어 설계부터 Petalinux 커널 드라이버, 그리고 최상위 PYNQ Python 라이브러리에 이르는 전체 포팅 과정이 성공적으로 완료되었음을 의미한다. 이는 포팅

작업의 궁극적인 검증이다.

결론 및 다음 단계

본 튜토리얼에서 제시한 절차를 통해 Zybo Z7-20 보드에 PYNQ v3.1을 성공적으로 포팅하고, 부팅 가능한 SD 카드 이미지를 생성하는 전 과정을 완료했다. 핵심은 PYNQ 버전에 맞는 정확한 AMD Xilinx 도구 체인(2024.1)을 사용하고, Docker 기반의 표준화된 빌드 환경을 구축하며, 각 단계(Vivado, Petalinux, PYNQ sdbuild)의 역할을 명확히 이해하는 것이었다.

이제 안정적이고 올바르게 구성된 PYNQ v3.1 기반 플랫폼이 준비되었으므로, 사용자는 이 기반 위에서 애플리케이션에 특화된 커스텀 하드웨어 오버레이를 자유롭게 설계하고 테스트할 수 있다. 일반적인 PYNQ 오버레이 개발 튜토리얼을 참조하여 가속화하고자 하는 기능을 IP로 구현하고, Python 환경에서 손쉽게 제어하는 PYNQ의 진정한 생산성을 경험할 수 있을 것이다.¹⁴

참고 자료

1. Vivado version for PYNQ v 3.0.1 on ZCU111 & ZCU208 - Support, 9월 26, 2025에 액세스, <https://discuss.pynq.io/t/vivado-version-for-pynq-v-3-0-1-on-zcu111-zcu208/7929>
2. PYNQ v3.0 (Belfast) Release - Announcements, 9월 26, 2025에 액세스, <https://discuss.pynq.io/t/pynq-v3-0-belfast-release/4765>
3. Releases · Xilinx/PYNQ - GitHub, 9월 26, 2025에 액세스, <https://github.com/Xilinx/PYNQ/releases>
4. Change Log — Python productivity for Zynq (Pynq) - Read the Docs, 9월 26, 2025에 액세스, <https://pynq.readthedocs.io/en/v3.1/changelog.html>
5. Current Status — Python productivity for Zynq (Pynq) - Read the Docs, 9월 26, 2025에 액세스, https://pynq.readthedocs.io/en/v3.1/pynq_remote/status.html
6. PYNQ 3.0.1 VirtualBox install and SD build - Learn, 9월 26, 2025에 액세스, <https://discuss.pynq.io/t/pynq-3-0-1-virtualbox-install-and-sd-build/4859>
7. Customize PetaLinux Image for AMD Xilinx Devices - MATLAB & Simulink - MathWorks, 9월 26, 2025에 액세스, <https://www.mathworks.com/help/soc/ug/customize-petalinux-image-xilinx.html>
8. PYNQ SD Card image — Python productivity for Zynq (Pynq) - Read the Docs, 9월 26, 2025에 액세스, https://pynq.readthedocs.io/en/latest/pynq_sd_card.html
9. Install Digilent's Board Files - Digilent Reference, 9월 26, 2025에 액세스, <https://digilent.com/reference/programmable-logic/guides/install-board-files>
10. Zybo Z7 Reference Manual - Digilent, 9월 26, 2025에 액세스, <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>
11. Zybo Z7 - Digilent Reference, 9월 26, 2025에 액세스,

- <https://digilent.com/reference/programmable-logic/zybo-z7/start>
12. Starter Lab With the Block Diagram, 9월 26, 2025에 액세스,
https://jodalyst.com/6s193/pynq_lab1/
 13. PYNQ Tutorial 1: GPIO - HackMD, 9월 26, 2025에 액세스,
<https://hackmd.io/@ween168/SyNPikUb1x>
 14. PYNQ Edition! Creating Custom Overlays - Hackster.io, 9월 26, 2025에 액세스,
<https://www.hackster.io/news/pynq-edition-creating-custom-overlays-8543f45eccb1>
 15. Overlay Tutorial — Python productivity for Zynq (Pynq) v1.0 - Read the Docs, 9월 26, 2025에 액세스,
https://pynq.readthedocs.io/en/v2.0/overlay_design_methodology/overlay_tutorial.html
 16. PYNQ overlay to accelerate some Python functions. - GitHub, 9월 26, 2025에 액세스,
<https://github.com/Pomiculture/PYNQ-overlay>
 17. Digilent/reVISION-Zybo-Z7-20 - GitHub, 9월 26, 2025에 액세스,
<https://github.com/Digilent/reVISION-Zybo-Z7-20>
 18. Quick Porting of PYNQ using Pre-built Images - Learn, 9월 26, 2025에 액세스,
<https://discuss.pynq.io/t/quick-porting-of-pynq-using-pre-built-images/1075>
 19. PYNQ v3.0.1 Build Failure for ZCU104 - Support, 9월 26, 2025에 액세스,
<https://discuss.pynq.io/t/pynq-v3-0-1-build-failure-for-zcu104/5398>
 20. Port PYNQ 3.01 to new board - Support, 9월 26, 2025에 액세스,
<https://discuss.pynq.io/t/port-pynq-3-01-to-new-board/5110>
 21. Adam Taylor's MicroZed Chronicles Part 158: Creating our own PYNQ hardware overlay, 9월 26, 2025에 액세스,
<https://adaptivesupport.amd.com/s/article/735388>